

Reporte de Proyecto

1. Introducción

El presente proyecto consiste en el desarrollo de un Motor de Inferencia Lógica como Servicio utilizando Node.js, Express y Prolog.

El sistema permite ejecutar consultas lógicas sobre una base de conocimiento mediante una API REST, integrando tecnologías modernas de desarrollo web con razonamiento simbólico.

La aplicación permite que un cliente envíe consultas lógicas mediante peticiones HTTP y reciba resultados inferidos automáticamente por el motor Prolog. Esto convierte al sistema en una plataforma flexible para representar conocimiento y realizar inferencias automáticas.

El objetivo principal del proyecto es demostrar la integración de múltiples paradigmas de programación dentro de una misma arquitectura de software moderna.

2. Explicación de Paradigmas de Programación Utilizados

El proyecto implementa distintos paradigmas de programación con el propósito de aprovechar las ventajas específicas de cada uno.

Programación Lógica

La programación lógica se implementa mediante Prolog, un lenguaje basado en hechos y reglas. Este paradigma permite representar conocimiento y realizar inferencias automáticas utilizando lógica formal.

Ejemplo:

```
contract(contract1).  
breach(contract1).
```

```
penalty_applicable(X) :-  
    contract(X),  
    breach(X).
```

En este caso, el sistema puede determinar automáticamente si una penalización aplica a un contrato determinado.

Programación Funcional

En JavaScript se utilizan funciones modulares para transformar datos y procesar información dentro del sistema.

El paradigma funcional permite:

- reutilización de código,
- modularidad,
- separación de responsabilidades,
- reducción de efectos secundarios.

Programación Asíncrona

Node.js implementa un modelo orientado a eventos y operaciones asíncronas.

El sistema utiliza:

- Promises
- async/await

Esto permite manejar múltiples solicitudes HTTP simultáneamente sin bloquear el servidor.

Ejemplo:

```
const result = await prologService.runQuery(query);
```

Arquitectura Modular

La aplicación está dividida en módulos independientes:

- rutas,
- controladores,
- servicios,
- lógica Prolog.

Esto facilita el mantenimiento y escalabilidad del sistema.

Tabla de Paradigmas Utilizados

Paradigma	Aplicación dentro del sistema
Lógico	Inferencias y reglas en Prolog
Funcional	Transformación de datos en JavaScript
Asíncrono	Manejo concurrente de peticiones
Modular	Separación de responsabilidades

3. Arquitectura del Sistema

La arquitectura del sistema se encuentra organizada por capas para mantener una clara separación de responsabilidades.

El flujo general del sistema es el siguiente:

1. El cliente envía una petición HTTP.
2. Express recibe la solicitud.
3. El controlador procesa la información.
4. El servicio Prolog ejecuta la consulta lógica.
5. El motor devuelve una inferencia.
6. El servidor responde en formato JSON.

Estructura de Carpetas

```
src/  
├─ app.js  
├─ routes/  
│   └─ query.routes.js  
├─ controllers/  
│   └─ query.controller.js  
├─ services/  
│   └─ prolog.service.js  
├─ prolog/  
│   └─ knowledgeBase.pl  
└─ utils/
```

Descripción de Componentes:

app.js

Punto de entrada principal del servidor Express.

routes/

Define los endpoints disponibles para la API REST.

controllers/

Gestiona las peticiones y respuestas HTTP.

services/

Contiene la lógica principal del sistema y la integración con Prolog.

prolog/

Contiene la base de conocimiento y las reglas lógicas.

utils/

Incluye funciones auxiliares y utilidades del sistema

4. Ejemplo de Ejecución

Base de Conocimiento

contract(contract1).

breach(contract1).

penalty_applicable(X) :-

contract(X),

breach(X).

Solicitud HTTP

POST /query

Content-Type: application/json

```
{  
  "query": "penalty_applicable(contract1)."  
}
```

Flujo de Ejecución

1. El usuario envía una consulta lógica.
2. Express recibe la petición.
3. El controlador valida el contenido.
4. El servicio carga la base de conocimiento.
5. Prolog ejecuta la inferencia.
6. El resultado se devuelve en formato JSON.

Respuesta del Sistema

```
{  
  "success": true,  
  "result": true  
}
```

5. Conclusiones y Extensiones Futuras

El proyecto permitió demostrar la integración efectiva entre programación lógica y tecnologías modernas de desarrollo web.

La utilización de Prolog proporciona capacidades avanzadas de razonamiento simbólico, mientras que Node.js y Express permiten exponer dichas funcionalidades mediante una API REST eficiente y escalable.

Asimismo, el proyecto evidenció la importancia de la programación asíncrona y la arquitectura modular en el desarrollo de sistemas mantenibles y organizados.

Extensiones Futuras

El sistema puede ampliarse mediante:

- soporte para múltiples bases de conocimiento,
- interfaz gráfica web,
- persistencia de datos,
- autenticación de usuarios,

- ejecución de múltiples consultas simultáneas,
- visualización gráfica de inferencias,
- almacenamiento dinámico de reglas.

Referencias

- Node.js Documentation
- Express.js Documentation
- Tau Prolog Documentation
- Bratko, I. *Prolog Programming for Artificial Intelligence*
- Richardson, L. *RESTful Web Services*